

Understanding `_build_features` (Shift, Lag, Rolling Mean)

Quick study PDF for Tausif • 2026-01-24

The function we are explaining

```
def _build_features(df: pd.DataFrame, timestamp_col: str, target_col: str) -> tuple[pd.DataFrame, list[str]]:
    df = df.copy()

    df["hour"] = df[timestamp_col].dt.hour
    df["day_of_week"] = df[timestamp_col].dt.dayofweek

    df["lag_1"] = df[target_col].shift(1)
    df["lag_24"] = df[target_col].shift(24)

    df["roll_mean_24"] = df[target_col].rolling(window=24, min_periods=24).mean().shift(1)

    df["y"] = df[target_col].shift(-1)

    feature_cols = ["hour", "day_of_week", "lag_1", "lag_24", "roll_mean_24"]
    return df, feature_cols
```

What is df?

df is a pandas DataFrame. Think of it like an Excel table in Python. Each row is one time step (one hour), and columns hold values like timestamp and flow_rate.

Example input columns (before we add features):

row	timestamp	flow_rate
0	2026-01-01 00:00	10
1	2026-01-01 01:00	12
2	2026-01-01 02:00	11
3	2026-01-01 03:00	13
4	2026-01-01 04:00	15
5	2026-01-01 05:00	14

How day_of_week is calculated

We take the timestamp for each row and ask: what weekday is this date? Pandas converts it to a number where Monday=0, Tuesday=1, ..., Sunday=6.

weekday	number (dayofweek)
Monday	0
Tuesday	1
Wednesday	2
Thursday	3
Friday	4
Saturday	5
Sunday	6

How shift works (visual)

shift(k) moves values in a column. Positive k moves values down (toward later rows). Negative k moves values up (toward earlier rows). Empty spots become NaN (missing).

row	time	flow_rate	flow_rate.shift(1)	flow_rate.shift(-1)
0	00:00	10	NaN	12
1	01:00	12	10	11
2	02:00	11	12	13
3	03:00	13	11	15
4	04:00	15	13	14
5	05:00	14	15	NaN

How lag_1 and lag_24 are calculated

lag_1 is the previous hour's value. We compute it with shift(1). So at time t, lag_1 holds flow_rate(t-1).

lag_24 is the value from 24 hours ago. We compute it with shift(24). Because your data is hourly, 24 rows back means 24 hours back (yesterday, same hour).

Why do lags help? They give the model memory. Time series often depend on recent history (what happened last hour and what happened yesterday).

What roll_mean_24 means

A rolling mean is a moving average. Imagine a window that always contains the last 24 flow_rate values. At each row, we take the average of the values inside the window and write it into a new column.

In code: `rolling(window=24, min_periods=24).mean()` means: use the last 24 rows; if we do not have 24 yet, output NaN; then take the average.

Rolling mean example with window=3 (easy to see)

We use 24 in your project, but 3 is easier to visualize. Here is `flow_rate` and a 3-hour rolling mean.

row	time	flow_rate	roll_mean_3 (no shift)
0	00:00	10	NaN (need 3 values)
1	01:00	12	NaN (need 3 values)
2	02:00	11	mean(10,12,11)=11.00
3	03:00	13	mean(12,11,13)=12.00
4	04:00	15	mean(11,13,15)=13.00
5	05:00	14	mean(13,15,14)=14.00

Why do we shift the rolling mean by 1 in your code?

Your code does `mean().shift(1)`. This pushes the rolling mean down one row so that, at time t , the rolling mean represents only the previous hours up to $t-1$. This matches how your forecasting loop computes `roll_mean_24` from history (it uses $t-24$ to $t-1$).

row	time	roll_mean_3 (no shift)	roll_mean_3.shift(1) meaning
2	02:00	11.00	NaN (moved down)
3	03:00	12.00	11.00 = mean(10,12,11) (past only)
4	04:00	13.00	12.00 = mean(12,11,13) (past only)
5	05:00	14.00	13.00 = mean(11,13,15) (past only)

What is `df['y']`?

`df['y']` is the target (the answer) the model should learn to predict. You set `df['y'] = flow_rate.shift(-1)`, which means: at time t , y holds `flow_rate(t+1)`. So each row becomes a training example: features at time t -> next hour's `flow_rate`.

What does the returned `df` look like?

The returned `df` includes the original columns plus the new feature columns and the target column y . Early rows have NaN because you cannot compute `lag_24` or `roll_mean_24` until you have enough history, and the last row has NaN for y because there is no next hour.

row	time	flow_rate	lag_1	lag_24	roll_mean_24	y
23	2026-01-01 23:00	121.348	120.202	NaN	NaN	122.168
24	2026-01-02 00:00	122.168	121.348	100.000	109.564	122.662
25	2026-01-02 01:00	122.662	122.168	101.782	110.487	122.863
26	2026-01-02 02:00	122.863	122.662	103.455	111.357	122.836

Key takeaways (one-liners)

- day_of_week: derived from the timestamp (Mon=0 ... Sun=6).
- lag_1: previous hour's flow_rate (shift down by 1).
- lag_24: yesterday's same-hour flow_rate (shift down by 24).
- roll_mean_24: average of the previous 24 hours (t-24 to t-1) because we shift by 1.
- y: next hour's flow_rate (shift up by 1).